CO5 – ASSESSMENT 3

MOCK INTERVIEW QUESTIONS

NAME: A.ARCHISHMAN

REGNO: 192425341

COURSE CODE: ITA0502

COURSE NAME: COMPUTER VISION

Q1. What is OpenCV, and why is it commonly used in computer vision applications?

Answer: OpenCV is an open-source computer vision library used for image/video processing, detection, tracking and feature extraction. It is popular because it is free, fast, cross-platform and supports Python and C++.

Q2. If you are given a folder containing hundreds of images, how would you design an OpenCV pipeline to read and process them efficiently?

Answer: List valid image files, read them one at a time using **cv2.imread()**, resize if necessary, preprocess, perform the required analysis and save the result. Avoid storing all images in memory; batch or parallel processing can improve speed.

Q3. What would you check if cv2.imread() returns None while reading an image?

Answer: Check the file path, file existence, extension/format, permissions and whether the image is corrupted. On Windows, use a correct path such as a raw string. Also print the absolute path to verify it.

Q4. Suppose a video is not opening using cv2.VideoCapture(). How would you systematically identify the problem?

Answer: Check the camera index or video path and test **cap.isOpened()**. Then check camera permissions, whether another application is using the camera, file/codec support and the value returned by **cap.read()**.

Q5. What is the difference between processing an image and processing a video stream in OpenCV?

Answer: An image is a single frame processed once. A video is a sequence of frames processed continuously, so it needs a loop, timing, resource management and usually tracking. Video processing also has real-time latency requirements.

Q6. If your video-processing application is dropping frames, what could be the reasons, and how would you improve its performance?

Answer: High resolution, heavy detection models, slow preprocessing and excessive I/O can cause frame drops. Resize frames, use an ROI, use lighter models, skip unnecessary frames and track between detections to improve performance.

Q7. How would you process only every 5th frame of a video? What are the advantages and disadvantages of doing this?

Answer: Use a frame counter and process when **frame_count % 5 == 0**. It reduces computation and improves speed, but fast motion may be missed and tracking accuracy can decrease.

Q8. Why might an image appear darker after reading it in OpenCV, and how would you correct the problem?

Answer: OpenCV reads color images in BGR order. Convert BGR to RGB using **cv2.cvtColor()** before displaying with RGB-based libraries. If the image is genuinely dark, use gamma correction, brightness adjustment or CLAHE.

Q9. If an OpenCV application has high processing latency, which stages of the pipeline would you investigate first?

Answer: Measure the time taken by acquisition, preprocessing, detection, feature extraction, tracking and visualization. Usually detection is a major bottleneck. Optimize the slowest stage using smaller inputs, ROI processing, lighter models or frame skipping.

Q10. How would you design a modular OpenCV application in which preprocessing, detection, tracking, and visualization are independent modules?

Answer: Create separate functions/classes such as **preprocess()**, **detect()**, **track()** and **visualize()**. Pass clear inputs and outputs between modules. This makes the system easier to test, debug and modify.

Q11. What is the difference between edge detection and corner detection? Give a practical application for each.

Answer: Edge detection finds boundaries where image intensity changes sharply; Canny can be used for lane or object-boundary detection. Corner detection finds points with strong changes in two directions; Shi-Tomasi can be used for feature tracking.

Q12. If a feature detector produces thousands of keypoints, how would you reduce the number of irrelevant features?

Answer: Keep only strong keypoints using a response threshold, apply non-maximum suppression or minimum-distance filtering, and restrict the ROI. For matching, use descriptor distance and ratio tests to remove weak matches.

Q13. How would you compare two images and determine whether they contain significant differences?

Answer: For aligned images, convert to grayscale, use **cv2.absdiff()**, threshold the difference and measure changed pixels/contours. For structural comparison, SSIM can be used. If images are misaligned, register them first.

Q14. Suppose feature matching works for two images taken from the same viewpoint but fails when one image is rotated. How would you improve the system?

Answer: Use rotation-invariant local features such as ORB or SIFT. Match descriptors using a ratio test and estimate a transformation such as homography with RANSAC when needed.

Q15. How would you design a system to track important features from one video frame to the next?

Answer: Detect strong features using Shi-Tomasi and track them between frames using Lucas-Kanade optical flow. Remove unreliable points and periodically detect new features when too few good points remain.

Q16. What are the major challenges in face detection when multiple people are present in a scene? How would you address them?

Answer: Challenges include overlapping faces, different sizes, pose, occlusion, poor lighting and blur. Use a multi-face detector, suitable confidence/scale settings, preprocessing and tracking to maintain detections across frames.

Q17. How would you design a simple face-recognition system using OpenCV? Explain the complete pipeline from image acquisition to identification.

Answer: Pipeline: **Capture** → **Face Detection** → **Crop/Align** → **Preprocess** → **Feature Extraction** → **Compare with Registered Database** → **Threshold** → **Identity/Unknown.** Multiple enrollment images and tracking improve reliability.

Pipeline: Capture → Preprocess → Detect → Track/Recognize → Decision → Output

Q18. A face-recognition system incorrectly identifies an unknown person as a known person. What changes would you make to reduce false recognition?

Answer: Use a stricter recognition threshold, better enrollment images, face-quality checks and multi-frame confirmation. Always allow an **Unknown** result instead of forcing the nearest known identity.

Q19. How would you improve face-recognition performance when images are captured under different lighting conditions?

Answer: Use CLAHE or histogram equalization, brightness normalization and face alignment. Include training/enrollment images under different lighting and use robust face embeddings with multi-frame confirmation.

Q20. What preprocessing techniques would you apply to a noisy scanned document before sending it to an OCR system?

Answer: Convert to grayscale, denoise using median/Gaussian filtering, correct illumination, apply adaptive/Otsu thresholding and use morphology. Deskewing and removing irrelevant regions also improve OCR accuracy.

Q21. If text in an image is rotated or captured at an angle, how would you prepare the image for OCR?

Answer: Estimate the skew angle and rotate the text to horizontal. If there is perspective distortion, detect the document corners and apply a four-point perspective transform. Then denoise and threshold before OCR.

Q22. How would you design a real-time system that detects moving objects in a surveillance video?

Answer: Use background subtraction such as MOG2/KNN, threshold and clean the foreground mask with morphology, then find contours and filter small regions. Track valid objects and process only the required ROI for real-time performance.

Pipeline: Capture → Preprocess → Detect → Track/Recognize → Decision → Output

Q23. Suppose a surveillance system generates many false motion detections because of moving trees and changing shadows. How would you reduce these false alarms?

Answer: Exclude trees using an ROI, enable shadow suppression, use minimum-area and morphological filtering, and require motion to persist across several frames. Adaptive background modelling also helps with gradual changes.

Q24. How would you design a people-counting system for a doorway? What problems could occur because of occlusion, people stopping, or changing direction?

Answer: Detect people, assign tracking IDs and use a virtual line. Count an ID only when its trajectory crosses the line in the required direction. Occlusion may lose IDs, stopping may create unstable tracks and direction changes may cause false crossings; robust tracking and trajectory history reduce these errors.

Q25. Imagine you are asked to develop a real-world Computer Vision application within one week using OpenCV. What application would you choose, and how would you design its complete pipeline? Explain why you selected each technique.

Answer: I would choose a **smart entrance people-counting system**. Pipeline: **Camera** → **ROI** → **Preprocessing** → **Person Detection** → **Tracking** → **Line Crossing** → **Count/Alert** → **Logging**. Lightweight detection and tracking make it achievable within one week and suitable for real-time operation.

Pipeline: Capture → Preprocess → Detect → Track/Recognize → Decision → Output